

Optimizing Streaming Tensor Decomposition on GPU

Wenqing Lin*

College of Artificial Intelligence
China University of
Petroleum-Beijing
Beijing, China
wenqing.lin@student.cup.edu.cn

Jianuo Sheng*

College of Artificial Intelligence
China University of
Petroleum-Beijing
Beijing, China
2023011609@student.cup.edu.cn

Shuqin Feng

College of Artificial Intelligence
China University of
Petroleum-Beijing
Beijing, China
2023011553@student.cup.edu.cn

Ming Dun

State Key Lab of Processors, Institute
of Computing Technology
Chinese Academy of Sciences
Beijing, China
dunming@ict.ac.cn

Huawei Cao

State Key Lab of Processors, Institute
of Computing Technology
Chinese Academy of Sciences
Beijing, China
caohuawei@ict.ac.cn

Qingxiao Sun[†]

China University of
Petroleum-Beijing
Beijing, China
Beihang University
Beijing, China
qingxiao.sun@cup.edu.cn

Abstract

Tensors represent multidimensional data and cover various areas of scientific computing. The Canonical Polyadic Decomposition (CPD) emerges to extract latent patterns from large but highly sparse tensors. In real-world scenarios, tensor slices often arrive dynamically over time in streaming form, making traditional CPD algorithms inefficient in processing the entire tensor at each time step. Streaming CPD processes tensor slices incrementally, exploiting a forgetting factor to adjust the weight of historical information to capture dynamics. Current optimizations mainly focus on CPU platforms, failing to meet the real-time processing requirements of modern applications. Efficiently deploying streaming CPD on GPU remains challenging due to frequent data transfers and memory operations throughout the complex workflow, as well as the intricate computational patterns of bottleneck operators.

To address these challenges, we propose ASTRA, a framework that optimizes the holistic streaming CPD process on GPU. At the system level, ASTRA integrates the memory pool and asynchronous strategy with compact storage format to reduce idle cycles and execution latency. At the operator level, ASTRA adopts dedicated techniques to customize the kernel implementations based on operator characteristics as well as an analytical model to adjust the parameters. Finally, ASTRA devises a similarity-aware updating mechanism to reduce the overall workload by skipping the calculations of highly similar adjacent time slices. The experimental results show that ASTRA achieves an average speedup of 1.71 $\times$ over the state-of-the-art methods with sufficient accuracy.

CCS Concepts

• **Computing methodologies** $\rightarrow$ **Massively parallel algorithms.**

*Both authors contributed equally to this work.

[†]Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License.
ICS '26, Belfast, United Kingdom

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2522-7/26/07

<https://doi.org/10.1145/3797905.3807865>

Keywords

GPU, Streaming Tensor Decomposition, Sparse Tensors

ACM Reference Format:

Wenqing Lin, Jianuo Sheng, Shuqin Feng, Ming Dun, Huawei Cao, and Qingxiao Sun. 2026. Optimizing Streaming Tensor Decomposition on GPU. In *2026 International Conference on Supercomputing (ICS '26)*, July 06–09, 2026, Belfast, United Kingdom. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3797905.3807865>

1 Introduction

Tensors, as generalizations of vectors and matrices in higher dimensions, serve as fundamental mathematical tools for representing multidimensional data. With the advancement of data acquisition technologies, tensor applications span from chemometrics [25] and signal processing [5] to recommendation systems [24] and image rendering [4]. Real-world tensors, such as user-item-time interactions or neuroscience signals [27], are not only massive in scale but also typically exhibit ultra-high sparsity.

Tensor decomposition has emerged to extract valuable latent patterns and structured information from complex multidimensional data [13]. The most popular method is Canonical Polyadic Decomposition (CPD) [29], which reveals intrinsic relationships by decomposing a tensor into a sum of rank components. Traditional CPD algorithms, such as solvers based on Alternating Least Squares (ALS) [9], are fundamentally designed for static, batch-oriented scenarios. They assume all data can be loaded into memory at once for computation. But in real-world scenarios, data often arrives dynamically over time in streaming form, such as sensor readings continuously generated from Internet of Things devices [26] or constantly updated user behaviors on social media [11].

Streaming CPD algorithms [33, 40] process continuously arriving data slices incrementally, utilizing a forgetting factor to adjust the weight of historical information to capture data dynamics. In a typical algorithm workflow, the primary computational burden is concentrated on three operators: Matricized Tensor Times Khatri-Rao Product (MTTKRP), Alternating Direction Method of Multipliers (ADMM), and Historical Information Update (HIST) [14, 32]. From our analysis, these operators account for more than 85% of the total

execution time on most public tensor datasets. Therefore, deeply optimizing these operators from a global workflow perspective is the key to improving streaming CPD performance.

Current optimization strategies for streaming CPD mainly focus on CPU platforms. Examples include the incremental tracking approach proposed by OnlineCP [44], the algorithm designed by CP-stream [28] to avoid linear growth in spatio-temporal overhead, and the method introduced by ALTO-stream [31] that addresses extreme sparsity through dynamic tensor linearization and time slicing. Despite significant algorithmic progress, performance gains are increasingly constrained by the low computational capabilities of CPUs, making it difficult to meet the growing demand for real-time processing in modern large-scale applications. Recent works have begun exploring implementing streaming CPD on GPU for further acceleration. For instance, OnlineGCP [22] computes streamed generalized CP decomposition, achieving cross-CPU/GPU portability via the Kokkos framework [7]. OFTD [42] utilizes implicit neural representations parameterized by neural networks to model streaming data. However, such implementations relying on high-level programming abstraction struggle to perform fine-grained optimizations tailored to specific hardware architectures.

Efficiently deploying streaming CPD on GPU remains a non-trivial task, as it faces three inherent challenges: 1) frequent GPU memory operations slow down the overall workflow and increase the risk of out-of-memory (OOM) errors; 2) time-consuming host-to-device (H2D) data transfers and their serial nature with kernel computations lead to idle GPU resources; 3) complex computational characteristics of bottleneck operators make them difficult to be fine-grained parallelized. We propose ASTRA, a framework that implements high-performance streaming CPD algorithm on GPU. To the best of our knowledge, this is the first work to systematically optimize the streaming CPD process in conjunction with customized GPU kernels. This paper makes the following contributions:

- We comprehensively analyze the performance bottlenecks of deploying streaming CPD on GPU, identifying key limiting factors and potential optimization opportunities.
- At the system level, we architect a unified GPU memory pool to obviate allocation and deallocation overheads. Additionally, we employ an asynchronous execution strategy with a compact tensor storage format to mask H2D latency.
- At the operator level, we design highly optimized kernels for MTTKRP, ADMM, and HIST. The kernel performance is further improved through warp shuffle and Tensor Core.
- We devise a similarity-aware adaptive updating mechanism to reduce computational workload by skipping highly similar time slices, and propose a lightweight analytical model for performance auto-tuning of bottleneck operators.
- We implement ASTRA, a framework integrating these optimizations. Experimental results demonstrate that ASTRA achieves an average speedup of 1.71× over state-of-the-art methods while maintaining comparable accuracy.

The rest of this paper is organized as follows. Section 2 and Section 3 present the background and motivation. Section 4 presents the methodology. Section 5 presents the evaluation results. Section 6 discusses the related work, and Section 7 concludes this paper.

2 Background

2.1 Tensor Notation

A tensor is a multi-dimensional array, a generalization of matrices and vectors. A higher-order tensor refers to a tensor with more than two dimensions, where mode- n denotes its n -th dimension. Higher-order tensors are widely used in fields like signal processing [21] and image rendering [4]. To clarify the upcoming discussion on CPD algorithms, we outline mathematical tensor notations in Table 1, which summarizes the key symbols and their definitions.

Table 1: List of tensor notations.

Symbol	Definition
$\mathcal{X}, \mathcal{Y}$	Multi-dimensional tensors.
N	Number of modes (dimensions) of a tensor.
I_n	Size of the n -th mode.
R	Rank of the CPD.
$\mathbf{A}^{(n)}$	Factor matrix for mode n , of size $I_n \times R$.
$\mathbf{S}$	Temporal factor matrix in streaming CPD.
$\mathbf{s}_t$	The t -th factor vector for the temporal mode.
$\mathbf{X}_{(n)}$	Mode- n matricization (unfolding) of a tensor $\mathcal{X}$.
$\ \mathcal{X}\ _F$	Frobenius norm of $\mathcal{X}$.
$(\cdot)^\dagger$	Moore-Penrose pseudoinverse.
$\llbracket \cdot \rrbracket$	Kruskal operator, e.g., $\mathcal{Y} \approx \llbracket \mathbf{A}^{(1)}, \dots, \mathbf{S} \rrbracket$
$\circ$	Outer product of vectors.
$\odot$	Khatri-Rao product (column-wise Kronecker product).
$\mathcal{X}_t$	Tensor slice arriving at time t .
T	Total number of time slices.
μ	Forgetting factor for historical information.
$\mathbf{M}^{(n)}$	Intermediate matrix from MTTKRP for updating mode n .
$\mathbf{G}^{(n)}$	Gram matrix for the least-squares solve for mode n .
$\mathbf{G}_{\text{hist}}$	Composite Gram matrix for the HIST operator.
$\mathcal{C}$	A set representing constraints (e.g., non-negativity).
$\mathbf{Z}, \mathbf{U}, \rho$	Auxiliary, dual variables, and penalty parameter in ADMM.
$\mathcal{R}_n(\mathbf{A})$	An optional regularization term for mode n .

2.2 Canonical Polyadic Decomposition

In data-intensive applications, multi-dimensional data is naturally represented as higher-order tensors. Tensor decomposition is a key tool for extracting latent features, reducing dimensionality, and uncovering patterns from such data. The Canonical Polyadic Decomposition (CPD) [13], also known as CANDECOMP/PARAFAC, decomposes a fixed N -mode tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times \dots \times I_N}$ into a sum of R rank-one components, as defined in (1):

$$\mathcal{X} \approx \sum_{r=1}^R \mathbf{a}_r^{(1)} \circ \mathbf{a}_r^{(2)} \circ \dots \circ \mathbf{a}_r^{(N)}. \quad (1)$$

For each mode n , the vectors $\mathbf{a}_r^{(n)}$ are collected as columns to form the factor matrix $\mathbf{A}^{(n)} = [\mathbf{a}_1^{(n)}, \mathbf{a}_2^{(n)}, \dots, \mathbf{a}_R^{(n)}] \in \mathbb{R}^{I_n \times R}$. For brevity, CPD is often written using the Kruskal operator notation as $\mathcal{X} \approx \llbracket \mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \dots, \mathbf{A}^{(N)} \rrbracket$. A significant advantage of CPD over other algorithms, such as Tucker decomposition [19], is its essential uniqueness under mild conditions. The standard approach for computing the CPD factors is to solve an optimization problem that minimizes the Frobenius norm of the reconstruction error (see (2)):

$$\min_{\mathbf{A}^{(1)}, \dots, \mathbf{A}^{(N)}} \frac{1}{2} \left\| \mathcal{X} - \llbracket \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(N)} \rrbracket \right\|_F^2. \quad (2)$$

Although this is a non-convex problem, *Alternating Least Squares* (ALS) [2, 8] has been shown to be an effective algorithm for solving it. The core idea of ALS is to fix all factor matrices except for one, solve the resulting convex least-squares problem for that single matrix, and repeat this process for every mode until convergence. The bottleneck operator within ALS is *Matricized Tensor Times Khatri-Rao Product* (MTTKRP) [15, 20, 29], which is defined as (3):

$$\mathbf{M}^{(n)} = \mathbf{X}_{(n)} \left(\bigodot_{k=1, k \neq n}^N \mathbf{A}^{(k)} \right), \quad (3)$$

where the intermediate matrix $\mathbf{M}^{(n)}$ is used to update the factor matrix $\mathbf{A}^{(n)}$ by solving a linear system.

The complete ALS update rule is given by (4):

$$\mathbf{A}^{(n)} \leftarrow \mathbf{M}^{(n)} \left(\bigodot_{k \neq n} (\mathbf{A}^{(k)\top} \mathbf{A}^{(k)}) \right)^{\dagger}. \quad (4)$$

For sparse tensors, explicitly forming and multiplying the matricized tensor $\mathbf{X}_{(n)}$ by the corresponding Khatri-Rao product is expensive and unnecessary. In most high-performance implementations, the product is computed *on-the-fly* for each non-zero element. For example, for a 3rd-order tensor $\mathcal{X}$ and a mode-1 MTTKRP, each non-zero element $x_{i,j,k}$ updates row i of $\mathbf{A}^{(1)}$ by using row j from $\mathbf{A}^{(2)}$ and row k from $\mathbf{A}^{(3)}$ to calculate the necessary row from the Khatri-Rao product.

2.3 Streaming CPD

Streaming CPD is designed for scenarios where tensor entries are observed sequentially over time. In the classical *single-aspect* streaming setting [12, 14], all tensor modes except the temporal mode remain fixed, while new data arrives along the temporal dimension. Figure 1 illustrates the streaming update process of a 3rd-order tensor. At each time step t , a new slice $\mathcal{X}_t \in \mathbb{R}^{I_1 \times \dots \times I_{N-1}}$ is observed and appended to the existing tensor along the temporal mode, forming the updated data $\mathcal{Y}_t = \mathcal{Y}_{t-1} \cup \mathcal{X}_t$.

Let $\mathcal{Y} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times T}$ denote an $(N+1)$ -way tensor, where N -way slices $\mathcal{X}_1, \dots, \mathcal{X}_T$ arrive sequentially over T batches. The objective is to find a rank- R CPD of $\mathcal{Y}$, as formulated in (5):

$$\min_{\{\mathbf{A}^{(n)} \in \mathbb{R}^{I_n \times R}\}, \mathbf{S} \in \mathbb{R}^{T \times R}} \frac{1}{2} \left\| \mathcal{Y} - \llbracket \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(N)}, \mathbf{S} \rrbracket \right\|_F^2, \quad (5)$$

where the streaming tensor $\mathcal{Y}$ is composed as a sequence of incoming slices [36], and each slice $\mathcal{X}_t$ is approximated by (6):

$$\mathcal{X}_t \approx \llbracket \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(N)}, \mathbf{s}_t \rrbracket, \quad \mathbf{s}_t \in \mathbb{R}^R. \quad (6)$$

This leads to the following incremental optimization problem over all observed slices, as defined in (7):

$$\min_{\{\mathbf{A}^{(n)}\}, \{\mathbf{s}_t\}} \sum_{t=1}^T \frac{1}{2} \left\| \mathcal{X}_t - \llbracket \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(N)}, \mathbf{s}_t \rrbracket \right\|_F^2. \quad (7)$$

In frameworks like CP-stream [28], the solution is incrementally updated by processing each newly arrived slice $\mathcal{X}_t$ sequentially. To maintain computational tractability and emphasize recent data, it does not re-process all historical slices. Instead, it approximates the past slices using the most recent factorization and incorporates a forgetting factor $\mu \in [0, 1]$. This leads to an objective function

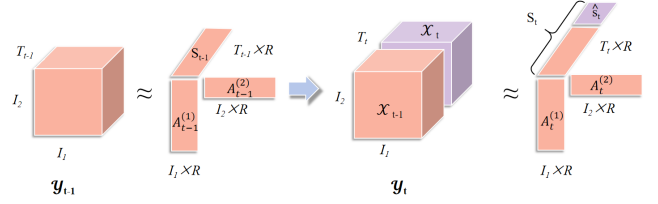


Figure 1: Single-aspect streaming CPD of a 3rd-order tensor.

that balances fitting the new slice with preserving past knowledge, weighted by μ , as formulated in (8):

$$\min_{\mathbf{A}^{(n)} \in \mathcal{C}} \frac{1}{2} \left\| \mathcal{X}_t - \llbracket \mathbf{A}_t^{(1:N)}; \mathbf{s}_t \rrbracket \right\|_F^2 + \frac{1}{2} \sum_{i=1}^{t-1} \mu^{t-i} \left\| \llbracket \mathbf{A}_{t-1}^{(1:N)}; \mathbf{s}_i \rrbracket - \llbracket \mathbf{A}_t^{(1:N)}; \mathbf{s}_i \rrbracket \right\|_F^2, \quad (8)$$

where $\mathcal{C}$ represents optional constraints like non-negativity. This objective is typically minimized using an ALS approach, which iteratively updates one factor matrix at a time.

For a given mode n , the ALS update first computes an intermediate matrix, which we denote as $\mathbf{M}_{\text{total}}^{(n)}$. This matrix aggregates gradient information from both the newly arrived data slice and all historical data. It is formed by summing two components: 1) the result of the MTTKRP operator on the current slice, and 2) contribution from past data calculated by the *Historical Information Update* (HIST) operator.

The MTTKRP operator computes the contribution from the current slice $\mathcal{X}_t$, and is the primary bottleneck in the ALS update, as defined in (9):

$$\mathbf{M}_{\text{current}}^{(n)} = \mathbf{X}_{t,(n)} \left(\bigodot_{\substack{k=1 \\ k \neq n}}^N \mathbf{A}^{(k)} \right). \quad (9)$$

To avoid the expensive re-computation over all historical data at each time step, the HIST operator efficiently approximates the gradient arising from the historical term (with forgetting factor μ) in the objective function (8). Specifically, the historical contribution $\mathbf{M}_{\text{hist}}^{(n)}$ is calculated as defined in (10):

$$\mathbf{M}_{\text{hist}}^{(n)} = \mathbf{A}_{t-1}^{(n)} \mathbf{G}_{\text{hist}}. \quad (10)$$

The HIST operator approximates the influence of historical data as a single matrix multiplication: it uses the factor matrix from the previous time step $\mathbf{A}_{t-1}^{(n)}$, and multiplies it by a composite Gram matrix $\mathbf{G}_{\text{hist}}$, which aggregates the historical interactions of all other modes. By summing the current and historical parts, the final intermediate matrix is derived as shown in (11):

$$\mathbf{M}_{\text{total}}^{(n)} = \mathbf{M}_{\text{current}}^{(n)} + \mathbf{M}_{\text{hist}}^{(n)}. \quad (11)$$

With the complete intermediate matrix $\mathbf{M}_{\text{total}}^{(n)}$ prepared, the factor matrix can be updated by solving a least-squares problem. For the more general case involving constraints such as non-negativity, the update is handled by the *Alternating Direction Method of Multipliers* (ADMM). ADMM introduces an auxiliary variable $\mathbf{Z}$ and

reformulates the problem into the following constrained optimization, as defined in (12):

$$\min_{\mathbf{A}, \mathbf{Z}} \frac{1}{2} \|\mathbf{M}_{\text{total}}^{(n)} - \mathbf{A}\mathbf{G}^{(n)}\|_F^2 + \mathcal{R}_n(\mathbf{Z}), \quad \text{s.t. } \mathbf{A} = \mathbf{Z}, \quad \mathbf{Z} \in \mathcal{C}, \quad (12)$$

where $\mathbf{G}^{(n)} = \odot_{k \neq n} (\mathbf{A}^{(k)\top} \mathbf{A}^{(k)})$.

ADMM iteratively updates the primary variable $\mathbf{A}$, the auxiliary variable $\mathbf{Z}$, and the dual variable $\mathbf{U}$ as detailed in (13):

$$\begin{aligned} \mathbf{A} &\leftarrow \left(\mathbf{M}_{\text{total}}^{(n)} + \rho(\mathbf{Z} - \mathbf{U}) \right) \left(\mathbf{G}^{(n)} + \rho \mathbf{I} \right)^{-1}, \\ \mathbf{Z} &\leftarrow \text{Proj}_{\mathcal{C}}(\mathbf{A} + \mathbf{U}), \\ \mathbf{U} &\leftarrow \mathbf{U} + \mathbf{A} - \mathbf{Z}. \end{aligned} \quad (13)$$

This scheme incorporates constraints by replacing the direct ALS update with optional ADMM in Algorithm 1.

Algorithm 1: Streaming CPD with optional ADMM.

Input: Rank R , initial factor matrices $\{\mathbf{A}_{t-1}^{(n)}\}_{n=1}^N$, forgetting factor $\mu \in [0, 1]$

```

1 for  $t \leftarrow 1, 2, \dots$  do
2   Receive new slice  $\mathcal{X}_t$ 
3   for  $n \leftarrow 1$  to  $N$  do
4      $\mathbf{M}_{\text{total}}^{(n)} \leftarrow$ 
        $\text{MTTKRP}(\mathcal{X}_t, \{\mathbf{A}_{t-1}^{(k)}\}_{k \neq n}) + \text{HIST}(\mathbf{A}_{t-1}^{(n)}, \dots)$ 
5      $\mathbf{G}^{(n)} \leftarrow \odot_{k \neq n} (\mathbf{A}_{t-1}^{(k)\top} \mathbf{A}_{t-1}^{(k)})$ 
6     if constraints  $\mathcal{C}$  exist then
7        $\mathbf{A}_t^{(n)} \leftarrow \text{ADMM}(\mathbf{M}_{\text{total}}^{(n)}, \mathbf{G}^{(n)}, \mathcal{C}, \mathcal{R}_n)$ 
8     else
9        $\mathbf{A}_t^{(n)} \leftarrow \arg \min_{\mathbf{A}} \|\mathbf{M}_{\text{total}}^{(n)} - \mathbf{A}\mathbf{G}^{(n)}\|_F^2 + \mathcal{R}_n(\mathbf{A})$ 
10     $\mathbf{A}_{t-1}^{(n)} \leftarrow \mathbf{A}_t^{(n)}$  // prepare for the next inner iteration
11  Apply forgetting factor  $\mu$  to historical information for the next time step
```

3 Motivation

We first naively implement the streaming CPD algorithm on GPU and evaluate it on public tensor datasets (*NIPS*, *Uber*, and *Chicago*) to thoroughly analyze the bottlenecks and optimization opportunities. The experiment setup is shown in Section 5.1.

3.1 Bottlenecks of Hotspot Operators

Figure 2 presents the time breakdown of streaming CPD across three datasets with varying densities. The core operators MTTKRP, ADMM, and HIST consistently account for the vast majority of the total execution time. Specifically, ADMM contributes the largest portion, ranging from 38.3% on the sparser *NIPS* dataset to 59.8% on the denser *Chicago* dataset, highlighting the increased computational cost of the solver on dense data. MTTKRP and HIST also consume a significant amount of time, accounting for 13.4% to 26.3% and 20.0% to 23.9%, respectively. Since these three operators constitute the primary cost, improving their efficiency is pivotal to boosting the performance.

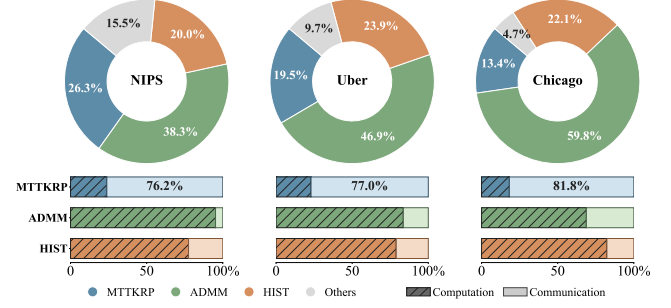


Figure 2: Time breakdown of streaming CPD across three tensor datasets with varying densities.

However, the fine-grained breakdown reveals that communication is the primary bottleneck. As shown in the horizontal bars, data transfer from CPU to GPU through PCIe consumes the vast majority of the execution time in the MTTKRP operator, ranging from 76.2% on *NIPS* to 81.8% on *Chicago*. This observation indicates that the system performance is heavily throttled by the constrained PCIe bandwidth during data preparation. Therefore, further optimizations are required to reduce communication overhead.

3.2 Fragmented Memory Allocation

The naive streaming CPD algorithm processes data incrementally, handling one time slice at a time. The program dynamically allocates GPU memory for intermediate data (such as factor matrices and partial results) at the start of processing each time slice and release it immediately upon completion. This approach causes fragmented memory allocation, resulting in extremely low efficiency when running on GPUs.

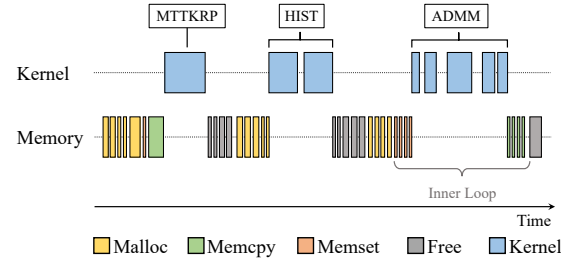


Figure 3: Fragmented memory allocation on the *Uber* dataset.

We employ NVIDIA Nsight Systems to profile a streaming CPD decomposition algorithm on the *Uber* dataset. As shown in Figure 3, numerous *Malloc*, *Memcpy*, *Memset*, and *Free* calls are interspersed among kernel executions. Repeatedly allocating and freeing small memory chunks leads to bandwidth underutilization and severe memory management cost. Moreover, such frequent memory requests force the GPU cores to remain idle most of the time due to host-device synchronization. The resulting fragmentation not only degrades end-to-end performance but also increases the risk of subsequent out-of-memory (OOM) failures.

3.3 Temporal Similarity in Stream Dynamics

Real-world streaming data is not chaotic and often exhibits significant temporal similarity between adjacent time slices. This similarity implies that previous decomposition results may change subtly after new slice insertion. Non-zero elements with spatial proximity often share similar latent features. Their contributions to factor updates are consequently highly correlated across time slices. When the spatial distribution and values of incoming tensors are extremely similar, the resulting MTTKRP and ADMM updates yield nearly identical factor gradients.

We track the overlap coefficient and pre-update RMSE for the *Uber* and *Patents* datasets across consecutive time steps. The pre-update RMSE represents the root mean square error when fitting the new data slice $\mathcal{X}_t$ using only historical factor matrices from the previous time step, which is formally defined as:

$$\text{RMSE} = \frac{\|\mathcal{X}_t - [\mathbf{A}_{t-1}^{(1)}, \dots, \mathbf{A}_{t-1}^{(N)}; \mathbf{s}_t]\|_F}{\sqrt{\text{NNZ}_t}}, \quad (14)$$

where NNZ_t denotes the number of non-zero elements in the current slice $\mathcal{X}_t$. As illustrated in Figure 4, both datasets exhibit a strong inverse correlation between structural continuity and initial error. In high-similarity intervals, high structural continuity aligns with minimal initial errors. This validates that the results from the previous time slice are sufficient to represent the current time slice. However, this consistency is not permanent, and failure to perform timely updates will lead to a collapse in accuracy.

Accurately judging temporal consistency requires capturing both structural and numerical changes. We adopt the overlap coefficient to quantify structural continuity and employ a norm-based check for value fluctuations. This enables a rigorous evaluation of the consistency between the incoming slice and the historical slices.

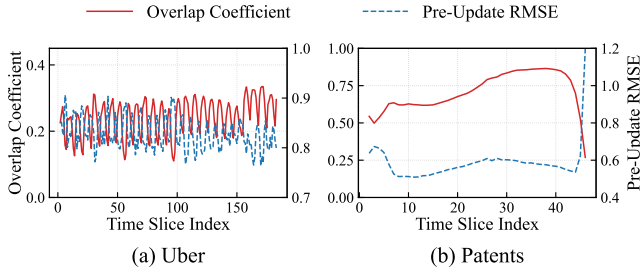


Figure 4: Correlation between overlap coefficient and pre-update RMSE on *Uber* and *Patents* datasets.

3.4 Parameter Sensitivity of Operators

Our optimized streaming CPD implementation exposes multiple parameters at the operator level that relate to underlying optimizations. Three primary parameters are: $\# \text{MTTKRP Chunks}$, which influences the overlap between computation and communication; $\# \text{ADMM Tiles}$, which balances shared memory usage and GPU occupancy; $\# \text{ADMM RHS per Block}$, which determines the parallel granularity of custom kernel functions.

However, the optimal configuration of these parameters is highly sensitive to the user option *Time Slice Width (TSW)*. This option

determines the batch size of input data processed per time step, directly impacting the GPU’s computational load and memory access patterns. We conduct experiments on the *Uber* dataset to evaluate how parameter configurations affect end-to-end performance. As shown in Figure 5, the optimal configuration (marked by the red circle) varies significantly with TSWs. For instance, a parameter configuration that performs well at $\text{TSW}=1$ might yield poor performance at $\text{TSW}=16$. The results indicate that in streaming processing scenarios, the optimality of parameter configurations exhibits high dynamism. A single parameter configuration struggles to maintain high performance across all workloads.

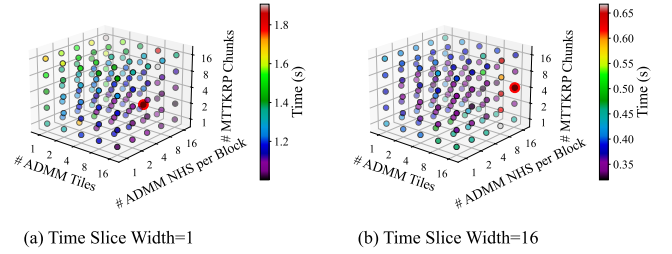


Figure 5: Streaming CPD time with different parameter configurations on the *Uber* dataset, where the time slice width is set to 1 and 16.

4 Methodology

4.1 Design Overview

In this section, we present a GPU-accelerated framework ASTRA for streaming CPD. As shown in Figure 6, ASTRA combines several system-level and operator-level optimizations to improve performance. At the system level, the framework employs a global GPU memory pool to eliminate frequent allocation overhead. To further address the bandwidth bottleneck, it introduces a compact tensor storage format alongside a stream pipeline design, effectively minimizing data transfer latency and kernel launch overheads. At the operator level, ASTRA meticulously optimizes the ADMM kernel with shared memory blocking and warp shuffle. For MTTKRP computation, the sparse tensor is partitioned to enable overlapping of data transfer and kernel computation. During the HIST stage, the Gram matrix updates are accelerated using mixed-precision Tensor Core while maintaining sufficient numerical accuracy.

Furthermore, ASTRA introduces an input-aware adaptive mechanism. It integrates a similarity-aware skipping strategy that utilizes temporal similarity to significantly boost processing efficiency. Simultaneously, the tuning module dynamically adjusts algorithm parameters based on the input tensor’s real-time characteristics, enabling efficient computation in a wide range of scenarios.

4.2 System-level Optimizations

We construct the ASTRA framework from the perspective of overall streaming CPD computation. The system-level optimizations mainly include GPU memory pool, compact tensor storage format, and overlapped streaming.

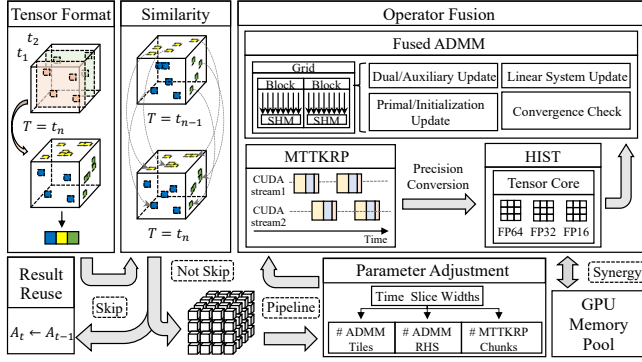


Figure 6: Overview of the ASTRA framework.

4.2.1 GPU Memory Pool. To effectively mitigate the fragmented memory operations mentioned in Section 3.2, ASTRA maintains a GPU memory pool, as illustrated in the Figure 7. Specifically, ASTRA pre-allocates a large contiguous space from GPU memory. Subsequent memory requests are satisfied by carving out smaller blocks from this pre-allocated pool. Released blocks are returned to the pool rather than directly freed. This approach effectively avoids redundant CUDA memory allocation and deallocation. To support multi-stream scenarios, the memory pool employs mutex locks to ensure thread safety, preventing concurrent modifications to the free block list that could cause data inconsistencies.

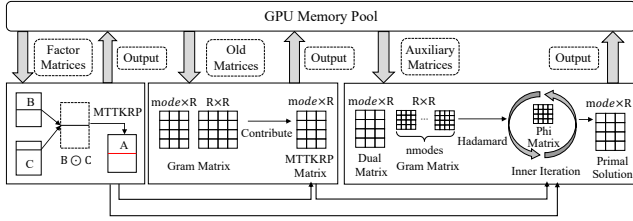


Figure 7: The role of GPU memory pool in ASTRA.

For the three core operators, ASTRA migrates all their memory allocation and deallocation operations to the internal memory pool. Taking MTTKRP as an example, factor matrices and output matrices across all modes are allocated and managed by the memory pool. To further reduce memory operations and data movement overhead, we perform operator fusion, enabling continuous execution on GPU and eliminating intermediate results. In traditional streaming CPD implementations, processing each time slice is divided into independent stages. Each stage requires Host-to-device (H2D) data transfers or device-side memory copies. By merging the stages into one single GPU operation, results generated by the MTTKRP stage can be directly reused by HIST and ADMM on GPU. This reduces H2D data transfers, with temporary matrices initialized and managed solely within the GPU.

4.2.2 Compact Tensor Storage Format. While the memory pool optimizes device-side allocation, PCIe transmission remains a critical bottleneck for streaming performance, as identified in Section 3.1. Existing formats struggle to meet the demands of streaming GPU

scenarios. To address this problem, ASTRA introduces the compact tensor storage format, which minimizes transmission latency by employing an input-aware, lightweight strategy that dynamically adjusts bit-widths without the need for complex global reordering.

Conventional formats impose a substantial memory footprint by utilizing fixed-width integers regardless of actual dimension sizes. As visualized in Figure 8, we first determine the minimum bit-width $b_m = \lceil \log_2 d_m \rceil$ required for each mode m . To map these compressed indices into the minimal number of 64-bit words, we employ a bitmask-based Dynamic Programming (DP) approach.

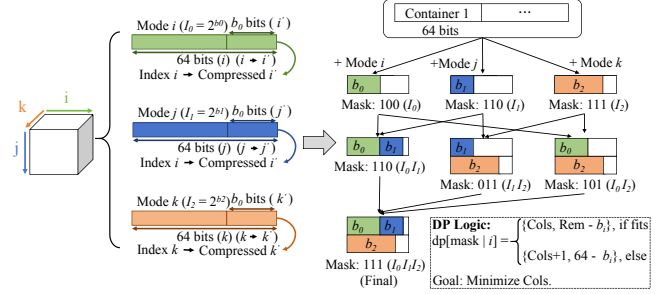


Figure 8: The dynamic packing logic of the compact tensor storage format.

Let S be the bitmask representing the subset of packed modes. The state $DP[S]$ is defined as a tuple (k, r) , where k denotes the number of 64-bit columns used, and r denotes the remaining bits in the current last column. For a transition from state S to $S \cup \{m\}$, the recurrence relation is defined as:

$$\mathcal{T}(S \cup \{m\}) = \begin{cases} (k, r - b_m) & \text{if } r \geq b_m, \\ (k + 1, 64 - b_m) & \text{if } r < b_m. \end{cases} \quad (15)$$

The algorithm traverses the state space to find a permutation that minimizes k primarily and maximizes r secondarily. This mathematically guarantees the most compact memory layout for arbitrary tensor topology.

On the host, ASTRA coordinates multi-threaded packing to execute vectorized bitwise operations and writes the compressed temporal tensor into pinned memory. The pinned memory maximizes PCIe throughput via direct memory access mechanism. To minimize preprocessing overhead, ASTRA caches packed metadata in GPU constant memory. On the device, the kernels fetch coalesced 64-bit words, and then reconstruct indices within the register file. This strategy replaces expensive global memory accesses with bitwise instructions, significantly boosting arithmetic intensity.

4.2.3 Overlapped Streaming. ASTRA employs a multi-stream execution strategy. By analyzing the data dependencies within the update rules, we identified opportunities to execute independent tensor operations concurrently.

Taking the HIST stage as an example, ASTRA decomposes the Gram matrix update process into independent contributions from each mode. The composite history term G_{hist} is updated as:

$$G_{\text{hist}} = G_{\text{old}} \odot \prod_{n \neq m} H^{(n)}, \quad \text{where } H^{(n)} = A^{(n)\top} A^{(n)}. \quad (16)$$

Since each $\mathbf{H}^{(n)}$ depends only on the factor matrix for mode n , its computation is independent and can be executed in parallel. Whenever the factor matrix $\mathbf{A}^{(n)}$ for a mode is ready, its Gram computation is immediately launched on a corresponding CUDA stream. This allows the matrix multiplications to be overlapped with the data transfer of other modes, effectively hiding the communication overhead.

After each mode's computation completes, a completion event is recorded on its respective CUDA stream for subsequent synchronization. The main stream waits for all completion events, ensuring each $\mathbf{H}^{(n)}$ is correctly accumulated into the shared Gram matrix $\mathbf{G}_{\text{hist}}$. Finally, global synchronization is invoked to complete the update of the Gram matrix.

4.3 Operator-level Optimizations

For the bottleneck operators in streaming CPD, we customize the optimizations to fit the GPU architecture, as described below.

4.3.1 MTTKRP. In streaming scenarios, the number of non-zero elements processed at each time step is relatively small compared to the model scale. Consequently, the primary performance bottleneck lies in the latency of transferring the irregular indices and values of the input sparse tensor slice $\mathbf{X}$ from the host to the device.

To mitigate this data movement overhead, ASTRA implements a fine-grained pipelining strategy. The input tensor $\mathbf{X}$ is partitioned into multiple chunks based on the number of non-zero elements to ensure load balancing:

$$\mathbf{X} = \bigcup_{k=1}^K \mathbf{X}_k, \quad \text{where } |\mathbf{X}_k| \approx \frac{|\mathbf{X}|}{K}. \quad (17)$$

Each chunk $\mathbf{X}_k$ is processed independently via a unified pipeline. ASTRA asynchronously transfers the value and index arrays of a chunk to the device and immediately launches the MTTKRP kernel for that chunk upon transfer completion. These tasks are assigned to multiple CUDA streams in a round-robin fashion. This design effectively overlaps the Host-to-Device (H2D) data transfer of the $(k+1)$ -th chunk with the GPU computation of the k -th chunk, thereby hiding the transmission latency.

To prevent race conditions during these parallel updates, our implementation employs atomicAdd operations within the CUDA kernels. This is combined with a *rank-split strategy* to minimize atomic contention and ensure the thread-safe accumulation of the intermediate matrix $\mathbf{M}^{(n)}$.

4.3.2 ADMM. ASTRA maximizes GPU efficiency by fusing all ADMM phases into a single monolithic kernel, as shown in Figure 9. This design processes right hand sides (RHSs) in batches while prefetching Cholesky factors and dual variables into shared memory to minimize global memory latency.

Since the coefficient matrix $(\mathbf{G}^{(n)} + \rho\mathbf{I})$ in Equation (13) is symmetric positive-definite, we compute its Cholesky decomposition $(\mathbf{G}^{(n)} + \rho\mathbf{I}) = \mathbf{L}\mathbf{L}^T$, where $\mathbf{L}$ is the lower-triangular matrix used in Algorithm 2.

The linear system solution employs the tiled parallel strategy described in Algorithm 2, where thread blocks independently process batches of right-hand side (RHS) columns, and threads within each block parallelize row-wise matrix computations. In forward

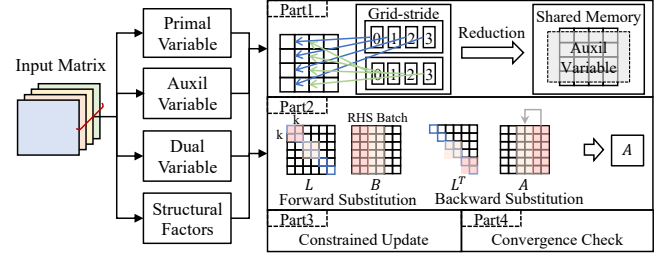


Figure 9: Fused ADMM kernel structure.

substitution (Lines 2 to 5), the kernel solves diagonal blocks and broadcasts the results via shared memory. This enables parallel updates of trailing submatrices, effectively converting serial substitutions into computationally intensive matrix multiplications. Similarly, backward substitution (Lines 6 to 9) uses this tiled structure to accumulate contributions from solved lower blocks, exposing concurrency across both matrix rows and RHS batches.

Following the linear solver, the kernel executes element-wise updates for primal, auxiliary, and dual variables using grid-stride loops. To support convergence assessment, diagnostic statistics are rapidly aggregated via warp-level shuffles and shared memory reductions, then committed to global memory using minimal atomic adds. Finally, a designated device thread evaluates the stopping criterion, eliminating unnecessary H2D communication.

Algorithm 2: TRSM design with a tiling approach.

Input: lower-triangular matrix $\mathbf{L} \in \mathbb{R}^{N \times N}$, matrix $\mathbf{B} \in \mathbb{R}^{N \times M}$, tile dimension TileDim

Output: The final solution $\mathbf{A} \in \mathbb{R}^{N \times M}$, which overwrites matrix $\mathbf{B}$

/ Phase 1: Forward Substitution (Solves $\mathbf{L}\mathbf{Y} = \mathbf{B}$, with result $\mathbf{Y}$ overwriting $\mathbf{B}$) */*

// Operations on the M columns of B are parallelized based on RHSs.

1 **for** $k = 0$ **to** $N/\text{TileDim} - 1$ **do**

2 $\mathbf{Y}_{k,:} \leftarrow \mathbf{L}_{k,k}^{-1} \mathbf{B}_{k,:}$;

3 **for** $i = k + 1$ **to** $N/\text{TileDim} - 1$ **do**

4 $\mathbf{B}_{i,:} \leftarrow \mathbf{B}_{i,:} - \mathbf{L}_{i,k} \mathbf{Y}_{k,:}$;

5 $\mathbf{B}_{k,:} \leftarrow \mathbf{Y}_{k,:}$;

/ Phase 2: Backward Substitution (Solves*

*$\mathbf{L}^T \mathbf{A} = \mathbf{Y}$, with result $\mathbf{A}$ overwriting $\mathbf{Y}$) */*

// Column-wise operations are again parallelized based on RHSs.

6 **for** $k = N/\text{TileDim} - 1$ **to** 0 **do**

7 $\mathbf{B}'_{k,:} \leftarrow \mathbf{B}_{k,:} - \sum_{j=k+1}^{N/\text{TileDim}-1} \mathbf{L}_{j,k}^T \mathbf{B}_{j,:}$;

8 $\mathbf{A}_{k,:} \leftarrow (\mathbf{L}_{k,k}^T)^{-1} \mathbf{B}'_{k,:}$;

9 $\mathbf{B}_{k,:} \leftarrow \mathbf{A}_{k,:}$;

4.3.3 HIST. During the HIST stage, the most computation-intensive task is calculating the Gram matrix $\mathbf{H}^{(n)} = \mathbf{A}^{(n)\top} \mathbf{A}^{(n)}$ for each

mode. To accelerate this critical operator, ASTRA employs a dedicated mixed-precision scheme leveraging Tensor Cores.

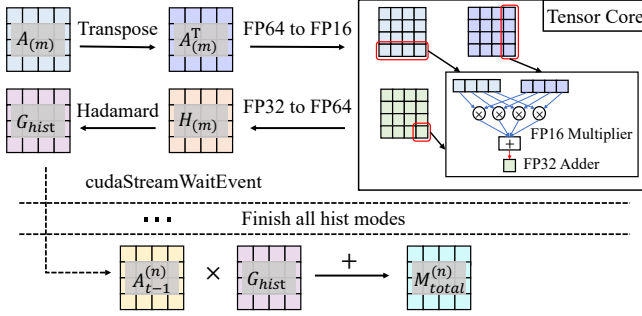


Figure 10: Gram matrix updates with Tensor Core on GPU.

As shown in Figure 10, for a specific mode assigned to a given stream, ASTRA first asynchronously transfers the factor matrix $A^{(n)}$ from the host to the device. To utilize Tensor Cores, the matrix is converted from double precision (FP64) to half precision (FP16). Subsequently, ASTRA invokes the `cublasGemmEx` interface with the `CUBLAS_TENSOR_OP_MATH` mode enabled. The Tensor Cores efficiently perform the self-matrix multiplication ($A^{(n)T} A^{(n)}$) in FP16 and store the result in single precision (FP32). Finally, the resulting $H^{(n)}$ is converted back to FP64 and accumulated into the global Gram matrix via a Hadamard element-wise product.

ASTRA substantially increases operator-level throughput and reduces the time required for Gram matrix updates, simultaneously maintaining sufficient numerical accuracy to ensure streaming CPD convergence (See Section 5.4). The overhead for precision conversion is negligible, accounting for less than 0.5% of the total HIST latency on large-scale datasets like *Flickr* and *Delicious*.

4.4 Similarity-Aware Adaptive Updating

As analyzed in Section 3.4, real-world streaming tensors often exhibit significant temporal locality. ASTRA introduces an adaptive mechanism to leverage this locality, effectively reducing computational overhead by reusing decomposition results when the system state remains relatively consistent. However, exact similarity detection relying on sorting operations incurs substantial latency.

Traditional similarity detection methods typically require K independent hash functions, leading to $O(K \cdot \text{NNZ})$ complexity. To enable high-throughput processing, ASTRA adopts the *One Permutation Hashing* (OPH) scheme proposed by Li *et al.* [16]. OPH utilizes a single permutation function to map non-zero indices into a continuous hash space, which is subsequently partitioned into K disjoint buckets. This design effectively reduces the similarity detection complexity to $O(\text{NNZ})$ by constructing a compact sketch $S_t \in \mathbb{R}^K$ for the incoming slice X_t , as defined in (18), while maintaining only the minimum hash value within each bucket:

$$S_t[b] = \min\{h(\text{idx}) \mid \text{idx} \in X_t, h(\text{idx}) \equiv b \pmod{K}\} \quad (18)$$

Standard MinHash provides an unbiased estimator for the Jaccard similarity (mathematically defined as $J = \frac{|X_t \cap X_{t-1}|}{|X_t \cup X_{t-1}|}$). In our implementation, the Jaccard similarity is computed efficiently as

the collision probability of the hash sketches:

$$\hat{J} = \frac{1}{K} \sum_{b=1}^K \mathbb{I}(S_t[b] = S_{t-1}[b]) \quad (19)$$

However, we observe that Jaccard similarity is overly sensitive to fluctuations in data volume, often failing to capture subset relationships where structural patterns persist despite intensity changes. ASTRA utilizes the *Overlap Coefficient* ($\hat{O}$) as a robust metric for structural continuity. Since direct computation of overlap is expensive, we analytically derive it from the estimated Jaccard similarity $\hat{J}$ and the NNZ counts of adjacent slices ($|X_t|$ and $|X_{t-1}|$):

$$\hat{O}(X_t, X_{t-1}) = \frac{\hat{J} \cdot (|X_t| + |X_{t-1}|)}{(1 + \hat{J}) \cdot \min(|X_t|, |X_{t-1}|)} \quad (20)$$

This transformation allows us to quantify structural overlap with $O(1)$ additional overhead.

Relying solely on index comparison is insufficient if tensor values change drastically. ASTRA employs a Dual-Check Mechanism to evaluate system stability. We consider the current time slice consistent with the historical state and eligible for factor reuse only if it satisfies high structural overlap and low numerical fluctuation:

$$\hat{O}(X_t, X_{t-1}) > \tau_{sim} \quad \wedge \quad \frac{||X_t||_F - ||X_{t-1}||_F|}{||X_{t-1}||_F} < \tau_{diff} \quad (21)$$

where $\|\cdot\|_F$ denotes the Frobenius norm.

If the stability check passes, ASTRA adopts a result reuse strategy to bypass the entire computational process for the current time slice. This optimization enables the system to skip the time-consuming MTTKRP-HIST-ADMM process. To maintain consistency with the streaming objective while avoiding heavy tensor-matrix operations, the factor matrices are directly inherited from the previous state. However, the optimization objective requires historical information to decay exponentially via μ . Therefore, the historical Gram matrix G_{old} undergoes a lightweight scalar decay:

$$A_t^{(n)} \leftarrow A_{t-1}^{(n)} \quad \text{and} \quad G_{old} \leftarrow \mu \cdot G_{old} \quad (22)$$

This targeted update can ensure that the historical regularization remains correctly scaled for subsequent non-skipped slices.

4.5 Input-aware Parameter Adjustment

Real-world tensors exhibit diversity and irregularity in their scale, density, and distribution of non-zero elements. In particular, the dynamic arrival of tensor slices in streaming scenarios makes the situation even more complicated. This renders the ASTRA framework highly sensitive to the selection of three critical runtime parameters: # MTTKRP Chunks (N_{chunk}), # ADMM Tiles (N_{tile}), and # ADMM RHS per Block (N_{rhs}). For MTTKRP optimizations, N_{chunk} indicates the number of chunks for each tensor slice. For ADMM optimizations, N_{tile} specifies the side dimension of the square tiles, while N_{rhs} is the number of RHSs processed within each thread block. Manually tuning the parameters at each time step is impractical. Therefore, ASTRA employs an analytical model that obtains near-optimal parameter configurations based on tensor properties and hardware constraints.

The analytical model requires the following input features: the number of non-zero elements (NNZ), the sparsity (σ), and the

time slice width (TSW). Note that the first two features are available when loading the incoming tensor slice and do not require additional analysis. It also considers key hardware specifications from the target GPU architecture, namely the total number of physical 32-bit registers per Streaming Multiprocessor (R_{total}). Since the streaming CPD algorithm adopts FP64 precision, each element occupies two 32-bit registers.

4.5.1 ADMM Tuning. The parameter $NRHS$ controls the thread parallelism within each thread block. The model uses a dual-sided principle to determine this value. For tensors meeting the Condition that being either ultra-large ($NNZ > 50 \times 10^6$) or ultra-sparse ($\sigma < 10^{-5}$), high thread parallelism is key to masking memory latency and saturating GPU capability. In this case, the model assigns a fixed high value of half a warp for maximum performance. For all other cases, the model employs a heuristic formula to balance thread parallelism and cache efficiency. The $NRHS$ is calculated as:

$$NRHS = \begin{cases} \frac{warp_size}{2} & \text{if Condition} \\ \text{Clamp}\left(2^{\text{round}\left(\log_2\left(\frac{0.6}{\sigma^{0.25}}\right)\right)}, 1, \frac{warp_size}{2}\right) & \text{else} \end{cases} \quad (23)$$

where $\text{Clamp}(x, \min, \max)$ restricts $x \in [\min, \max]$.

The $NTile$ determines the workload assigned to each thread. Its optimal value is constrained by per-thread register availability and is inversely proportional with $NRHS$. Specifically, the available register budget for FP64 computations is half the total number of 32-bit registers ($R_{total}/2$). This constraint is integrated into the formula as follows:

$$NTile = \text{Clamp}\left(2^{\text{round}\left(\log_2\left(\zeta \cdot \frac{R_{total}/2}{NRHS \cdot (1-\sigma)}\right)\right)}, 1, \frac{warp_size}{2}\right), \quad (24)$$

where the fitting constant ζ is set to 0.008 empirically.

4.5.2 MTTKRP Tuning. Regarding the chunking strategy, the parameter $NChunk$ influences the overlap between computation and communication. Our investigation identifies three distinct regimes based on the tensor's scale and sparsity. We define three conditions: 1) ultra-large ($NNZ > 50 \times 10^6$) or ultra-sparse ($\sigma < 10^{-5}$); 2) moderately-sized ($nnz \leq 50 \times 10^6$) and moderately-sparse ($10^{-5} \leq \sigma \leq 0.005$); 3) moderately-sized ($nnz \leq 50 \times 10^6$) and low-sparse ($\sigma > 0.005$). For Condition 1, the performance is primarily dictated by streaming throughput. For Condition 2 and Condition 3, the primary factors are kernel launch overhead and task pipelining efficiency, respectively. The $NChunk$ is calculated as:

$$NChunk = \begin{cases} 2 & \text{if Condition 1} \\ \text{round}\left(\frac{TSW}{8}\right) & \text{if Condition 2} \\ \text{Clamp}\left(\text{round}\left(\log_2(TSW + 1)\right), 1, 4\right) & \text{if Condition 3} \end{cases} \quad (25)$$

The input-aware analytical model provides portability across GPU architectures. It can determine near-optimal parameter configuration at runtime while avoiding costly search.

5 Evaluation

5.1 Experiment Setup

5.1.1 System Configurations. The hardware specifications are presented in Table 2. The experiments are conducted on Ubuntu 18.04.6 LTS with GCC v7.5.0 and NVCC v12.6.

Table 2: Hardware specifications.

	CPU	GPU
Model	Intel Xeon Gold 6230R	NVIDIA A100
Frequency	2.1GHz Base (4.0GHz Turbo)	1.41GHz
Cores	52	6912 (108 SMs)
Cache	128KB L1, 2MB L2, 71.5MB L3	192KB L1 (per SM), 80MB L2
Memory	377GB DDR4	80GB HBM2
Bandwidth	140.7GB/s	1,935GB/s

5.1.2 Implementations. To evaluate the performance of ASTRA, we benchmark it against two CPU-based methods (CP-stream [30] and ALTO-stream [31]) and one GPU-based method (OnlineGCP [22]). To ensure fairness, all methods employ official implementations and standardize algorithmic parameters, including rank (16), forgetting factor (0.99), iteration count (2), and constraints (non-negative). We set the number of outer iterations to 2 because the fitting error generally tends to stabilize at this point. In addition, the CPU-based methods utilize all available cores for calculations.

5.1.3 Datasets. We utilize several real-world sparse tensors obtained from the FROSTT repository [27]. Table 3 details the tensor properties, including order, dimensions, nnz, and density. The underlined dimension represents the time dimension.

Table 3: Tensors used for evaluation.

Tensor	Order	Dimensions	nnz	Density
<i>Vast</i>	3	<u>165K</u> $\times$ 11K $\times$ 2	26M	6.9×10^{-3}
<i>Patents</i>	3	<u>46</u> $\times$ 239.2K $\times$ 239.2K	3.6B	1.4×10^{-3}
<i>Uber</i>	4	<u>183</u> $\times$ 24 $\times$ 1140 $\times$ 1717	3M	3.9×10^{-4}
<i>NIPS</i>	4	2K $\times$ 3K $\times$ 14K $\times$ <u>17</u>	3M	1.8×10^{-6}
<i>Chicago</i>	4	<u>6.2K</u> $\times$ 24 $\times$ 77 $\times$ 32	5.3M	1.5×10^{-2}
<i>Enron</i>	4	6K $\times$ 6K $\times$ 244K $\times$ <u>1K</u>	54M	5.5×10^{-9}
<i>Flickr</i>	4	320K $\times$ 28M $\times$ 2M $\times$ <u>731</u>	113M	1.1×10^{-14}
<i>Delicious</i>	4	533K $\times$ 17M $\times$ 3M $\times$ <u>1K</u>	140M	4.3×10^{-15}

5.2 End-to-end Performance

Figure 11 illustrates the end-to-end performance comparison of different streaming CPD methods. The time slice width (TSW) is set to 1, 4, and 16. Note that CP-stream and OnlineGCP do not support

scenarios where TSW is greater than 1. As observed, ASTRA generally outperforms the baseline (ALTO-stream) across all datasets, demonstrating consistent computational acceleration.

When TSW is set to 1, ASTRA achieves significant speedups ranging from 1.10× to 1.77× over the baseline. On large-scale datasets such as *Flickr* and *Delicious*, ASTRA demonstrates exceptional efficiency with speedups of 1.77× and 1.64×, respectively. For relatively dense tensors like *Chicago*, ASTRA secures a 1.88× speedup by leveraging shared memory blocking in fused ADMM kernels to minimize memory latency. On smaller datasets like *NIPS* and *Uber*, the compact tensor format proves critical in reducing PCIe latency, resulting in speedups of 2.04× and 1.85×. ASTRA's efficiency at this granularity is further enhanced by its similarity-aware adaptive updating mechanism. For datasets with high temporal locality like *Uber* and *Patents*, this strategy identifies and skips 67.76% and 60.87% of redundant time slices, respectively, yielding speedups of 2.38× and 3.24× compared to execution without skipping.

As TSW increases to 4 and 16, large-scale tensors show improved scalability as larger batches better amortize kernel overheads. For instance, *Flickr* reaches a 1.96× speedup and *Enron* achieves a 2.20× speedup over the baseline at TSW=16. Small datasets also benefit from this batching effect; *NIPS* achieves a 2.42× speedup as increased computational intensity masks launch latencies. This coarser granularity smooths out short-term fluctuations, significantly enhancing ASTRA's detection of structural similarity. On the *Vast* and *Chicago* datasets, the skipping rates surge to 62.67% and 77.24% at TSW=4, driving speedups of 2.58× and 3.63×, respectively.

In contrast, OnlineGCP stands as one of the few streaming CPD methods supporting GPU, yet its performance is constrained by the Kokkos framework and stochastic gradient descent algorithm. This approach incurs significant overhead from additional sampling and gradient computation, along with the risk of OOM failures (3 of 8 datasets). For the sake of brevity without affecting the analysis, the following contents only report the results with a TSW of 1.

5.3 Operator Performance

The performance comparison of core operators MTTKRP, ADMM, and HIST is presented in Table 4. Note that OnlineGCP adopts a different algorithm, resulting in inconsistent operator breakdown with other methods, thus not shown in the table.

For MTTKRP, the execution time of the GPU kernel integrated in ASTRA is short. Consequently, the performance is dominated by H2D tensor slice transfers, constrained by the PCIe bandwidth bottleneck. ASTRA minimizes data movement overhead by maintaining factor matrices resident in GPU memory and leveraging pipelined overlapping with a compact storage format. For instance, on the *Delicious* dataset, ASTRA achieves a 34.1× speedup over ALTO-stream. It also demonstrates advantages on datasets with high NNZ (indicating high data transfer overhead); on the *Patents* dataset, ASTRA reduces the time from 40.61s to 4.72s.

For the compute-intensive HIST and ADMM operators, ASTRA demonstrates robust performance gains across most datasets by synergizing mixed-precision Tensor Cores with fully fused kernels. On the *Flickr* dataset, HIST execution is accelerated by approximately 2.1×. Similarly, ASTRA achieves substantial speedups in ADMM; for instance, on *Delicious*, the iterative solution time is

drastically reduced from 3298.00s to 901.48s, showcasing the efficiency of GPU parallelism for large-scale factor updates. However, for datasets with extreme dimensionality ratios such as *Patents*, the minimal temporal dimension size $l = 46$ renders the ADMM updates memory-bound, resulting in a slowdown compared to the CPU baselines. Despite this latency, ASTRA still guarantees the shortest end-to-end execution time, outperforming ALTO-stream.

For CPU-based baselines, the total execution time is inflated by non-computational overheads. For instance, on the *Uber* dataset, the three core operators account for approximately 69.2% of the total runtime in ALTO-stream. The remaining time is consumed by auxiliary tasks, such as dynamic matrix resizing and memory initialization. In contrast, ASTRA migrates the preprocessing portion to GPU, ensuring end-to-end speedup despite performance variations in individual operators.

5.4 Convergence Analysis

We examine the fitting error of ASTRA to evaluate the impact of optimization strategies on numerical accuracy. Consistent with literature [31], we adopt the standard global relative fitting error as the primary metric, which is defined as:

$$\text{Fitting Error} = \frac{\|\mathcal{Y} - \llbracket \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(N)}, \mathbf{S} \rrbracket\|_F^2}{\|\mathcal{Y}\|_F^2} \quad (26)$$

where $\mathcal{Y}$ is the complete original tensor and $\llbracket \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(N)}, \mathbf{S} \rrbracket$ is the tensor reconstructed from the final factor matrices including the temporal factor $\mathbf{S}$.

Figure 12 shows the fitting error comparison of three streaming CPD methods across tensor datasets. The results indicate that ASTRA achieves significant acceleration without noticeably compromising the final decomposition quality.

5.5 Ablation Study

Table 5 validates the effectiveness of ASTRA's system components through a comprehensive ablation analysis. The GPU memory pool is critical for system stability. Disabling this pre-allocated management not only triggers out-of-memory errors on large-scale tensors but also causes a severe 10.58× performance degradation on the dense *Chicago* dataset due to excessive memory allocation overheads. In terms of communication optimization, the compact tensor storage format excels on small-scale, bandwidth-sensitive tensors. Disabling this format achieves significant slowdowns of 1.46× and 1.41× on the *NIPS* and *Chicago* datasets, respectively. Although the highly optimized MTTKRP kernels leaves limited latency to be hidden, the pipeline mechanism remains effective. Disabling pipelining causes computation and memory access operations to be serialized, resulting in a slowdown of up to 1.22×.

5.6 Parameter Tuning Analysis

To evaluate the core parameters identified in Section 4.5, ASTRA integrates an analytical model for parameter tuning. Based on the analysis across datasets, the model accurately identifies the optimal configuration in 33.33% of cases. For the remaining suboptimal predictions, the average performance degradation is only 2.16% compared to the manually tuned best configuration. Even when the most critical parameter *NChunk* is mismatched (with 55.56%

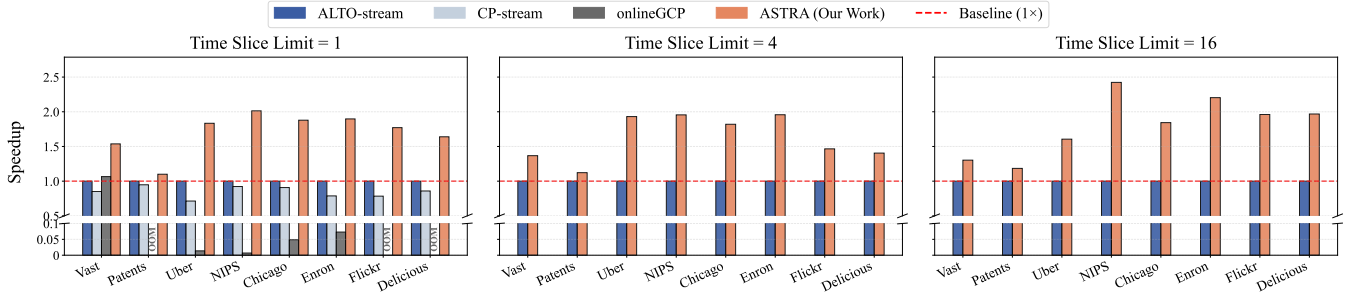


Figure 11: End-to-end performance comparison of different streaming CPD methods.

Table 4: Performance comparison of individual operators across all datasets (Best results are highlighted in bold).

Dataset	Total Time (s)			MTTKRP (s)			HIST (s)			ADMM (s)		
	ALTO-stream	CP-stream	ASTRA	ALTO-stream	CP-stream	ASTRA	ALTO-stream	CP-stream	ASTRA	ALTO-stream	CP-stream	ASTRA
<i>Vast</i>	1353.27	1589.03	880.36	51.04	111.88	19.13	323.42	453.97	168.21	608.53	660.36	293.28
<i>Patents</i>	130.94	138.16	119.05	40.61	73.73	4.72	5.12	8.99	4.31	12.82	13.52	45.32
<i>Uber</i>	1.20	1.68	0.65	0.18	0.72	0.03	0.21	0.29	0.15	0.44	0.46	0.30
<i>NIPS</i>	0.57	0.62	0.28	0.05	0.14	0.01	0.10	0.15	0.06	0.25	0.20	0.09
<i>Chicago</i>	20.92	23.05	11.14	2.89	5.44	0.72	3.31	3.25	2.50	8.38	8.61	6.05
<i>Enron</i>	173.61	220.45	91.52	5.08	37.38	0.29	50.76	75.24	22.81	79.17	74.49	18.28
<i>Flickr</i>	5699.75	7270.32	3217.83	15.87	25.11	0.40	1631.96	2989.11	774.33	2859.08	3148.47	664.49
<i>Delicious</i>	7206.23	8394.55	4395.64	21.86	34.70	0.64	2265.53	4178.04	1069.05	3298.00	2655.07	901.48

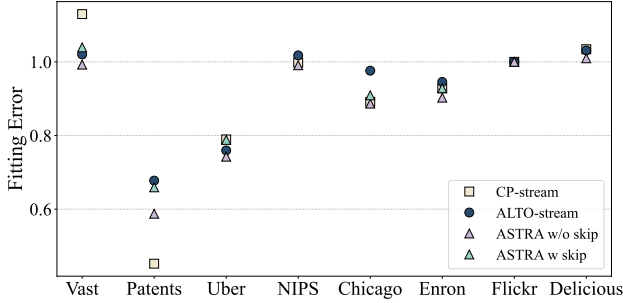


Figure 12: Fitting errors of streaming CPD methods.

Table 5: Ablation study of ASTRA components.

Dataset	Slowdown vs. ASTRA		
	w/o Memory Pool	w/o Pipeline	w/o Compact Format
<i>Vast</i>	OOM	1.10×	1.16×
<i>Patents</i>	OOM	1.06×	1.38×
<i>Uber</i>	1.80×	1.05×	1.35×
<i>NIPS</i>	1.28×	1.10×	1.46×
<i>Chicago</i>	10.58×	1.15×	1.41×
<i>Flickr</i>	OOM	1.04×	1.05×
<i>Delicious</i>	OOM	1.05×	1.06×
<i>Enron</i>	OOM	1.22×	1.08×

accuracy), the model still avoids severe slowdown by adapting the configuration to an appropriate range.

5.7 Overhead Analysis

We evaluate the runtime costs of ASTRA’s auxiliary components. The on-the-fly index decoding within the MTTKRP kernel is fully masked by memory and arithmetic latencies. It accounts for merely $0.23\mu\text{s}$ on *Chicago* and less than 0.01% of the runtime on *Uber*. The overhead for precision conversion in the HIST component is effectively amortized by the subsequent Tensor Core acceleration, consuming only 0.45% of the operator time on *Delicious*.

The similarity detection mechanism maintains low latency. For large-scale workloads like *Vast*, the check latency represents less than 0.1% of the total processing time. Even for *Patents*, which features the densest index space, the detection overhead is only 1.73 s. The analytical model for parameter tuning operates in the microsecond range ($0.5\mu\text{s}$ to $0.7\mu\text{s}$), which is six to eight orders of magnitude lower than the execution time. Collectively, these results verify that system-level overheads do not impose a significant impact on ASTRA’s performance.

6 Related Work

6.1 Streaming CPD Optimizations

Research on streaming tensor decomposition employs strategies ranging from incremental algebraic updates to randomized sketching [14, 30, 31, 39, 44]. On CPUs, methods like DAO-CP [33], SliceNStitch [14], and GOCPT [39] focus on adapting to time-varying or evolutionary tensor modes. Others optimize specific constraints:

CP-stream [28] manages linear growth, ALTO-stream [31] targets extreme sparsity via linearized formats, and ACP [1] utilizes randomized sketching for real-time tracking. Recent works also explore GPU acceleration. OnlineGCP [22] leverages Kokkos for cross-architecture portability, while OFTD [42] pioneers a continual learning paradigm using Implicit Neural Representations. As shown in Table 6, compared to these state-of-the-art methods, ASTRA exhibits superior adaptability by integrating auto-tuning with GPU-accelerated processing for time-sliced streams.

Table 6: Comparison of SOTA streaming CPD methods.

	GPU Support	Time Slices	Constraints	Auto-Tuning
SliceNStitch [14]	✗	✗	✗	✗
CP-stream [28]	✗	✗	✓	✗
DAO-CP [33]	✗	✗	✓	✗
ACP [1]	✗	✗	✗	✗
FTD [42]	✓	✗	✓	✗
ALTO-stream [31]	✗	✓	✓	✓
OnlineGCP [22]	✓	✗	✓	✓
ASTRA (ours)	✓	✓	✓	✓

6.2 Auto-tuning for Tensor Algebra

Automatic tuning of tensor computations primarily focuses on optimizing sparse storage formats and execution graphs to adapt to hardware characteristics [3, 6, 10, 17, 23, 34, 35, 38, 41, 43]. SpTFS [34] predicts the optimal storage format for MTTKRP computation. AlphaSparse [6] uses an operation graph-based method to automatically generate sparse matrix formats and SpMV kernels optimized for specific hardware. HP-CSF [18] further advances format-specific optimization by establishing parallel deployment rules and thread-level load balancing for the CSF format. WISE [41] introduces a “skewness-local sensitivity” feature set and combines it with machine learning models to predict the optimal SpMV kernel. AutoSparse [23] employs a dynamic computation graph-based DSL and reinforcement learning-guided search. Recent research extends these optimizations to GPU architectures. For instance, Wijeratne et al. [37] proposes a mode-specific layout and an adaptive partitioning mechanism that automatically toggles between index-based and non-zero-based distribution to match hardware parallelism.

While effective, these learning-based or search-intensive approaches incur severe preprocessing and runtime overheads. To avoid high latency in streaming CPD scenarios, we instead employ a lightweight yet effective analytical model for parameter tuning.

7 Conclusion

This paper addresses multiple challenges in deploying streaming CPD on GPU, including memory operations, data transfer, and operator parallelization. To this end, we propose ASTRA, which combines system-level optimizations with operator-level innovations, as well as a lightweight analysis model for time-slice-aware parameter adaptation. In addition, ASTRA devises a similarity-aware updating mechanism that skips the calculations of highly similar

adjacent temporal tensors to reduce the overall workload. The experimental results demonstrate that ASTRA achieves end-to-end acceleration over state-of-the-art methods on real-world datasets without sacrificing convergence accuracy.

Acknowledgments

This work is supported by National Natural Science Foundation of China (No. 62402525). Qingxiao Sun is the corresponding author.

References

- [1] Karim Abed-Meraim, Nguyen Linh Trung, Adel Hafiane, et al. 2023. Tracking online low-rank approximations of higher-order incomplete streaming tensors. *Patterns* (2023), 100763.
- [2] J Douglas Carroll and Jih-Jie Chang. 1970. Analysis of individual differences in multidimensional scaling via an N-way generalization of “Eckart-Young” decomposition. *Psychometrika* (1970), 283–319.
- [3] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. 2018. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 578–594.
- [4] Robert L Cook, Loren Carpenter, and Edwin Catmull. 1987. The Reyes image rendering architecture. *ACM SIGGRAPH Computer Graphics (ACM SIGGRAPH Comput. Graph.)* (1987), 95–102.
- [5] André LF De Almeida and Alain Y Kibangou. 2014. Distributed large-scale tensor decomposition. In *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 26–30.
- [6] Zhen Du, Jiajia Li, Yinshan Wang, Xueqi Li, Guangming Tan, and Ninghui Sun. 2022. Alphaspase: Generating high performance spmv codes directly from sparse matrices. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE, 1–15.
- [7] H Carter Edwards, Daniel Sunderland, Vicki Porter, Chris Amsler, and Sam Mish. 2012. Manycore Performance-Portability: Kokkos Multidimensional Array Library. *Scientific Programming (Sci. Program.)* (2012), 89–114.
- [8] Richard A Harshman et al. 1970. Foundations of the PARAFAC procedure: Models and conditions for an “explanatory” multi-modal factor analysis. *UCLA working papers in phonetics (UCLA Work. Pap. Phonetics)* (1970), 84.
- [9] Ahmed E Helal, Jan Laukemann, Fabio Checconi, Jesmin Jahan Tithi, Teresa Ranadive, Fabrizio Petrini, and Jeewhan Choi. 2021. Alto: Adaptive linearized storage of sparse tensors. In *Proceedings of the 35th ACM International Conference on Supercomputing (ICS)*. ACM, 404–416.
- [10] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. 2019. TASO: optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 47–62.
- [11] Alexandros Karatzoglou, Xavier Amatriain, Linas Baltrunas, and Nuria Oliver. 2010. Multiverse recommendation: n-dimensional tensor factorization for context-aware collaborative filtering. In *Proceedings of the fourth ACM conference on Recommender systems (RecSys)*. ACM, 79–86.
- [12] Hiroyuki Kasai and Bamdev Mishra. 2016. Low-rank tensor completion: a Riemannian manifold preconditioning approach. In *International conference on machine learning (ICML)*. PMLR, 1012–1021.
- [13] Tamara G Kolda and Brett W Bader. 2009. Tensor decompositions and applications. *SIAM review (SIAM Rev.)* (2009), 455–500.
- [14] Taehyung Kwon, Inkyu Park, Dongjin Lee, and Kijung Shin. 2021. Slicenstitch: Continuous cp decomposition of sparse tensor streams. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 816–827.
- [15] Jiajia Li, Jimeng Sun, and Richard Vuduc. 2018. HiCOO: Hierarchical storage of sparse tensors. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE, 238–252.
- [16] Ping Li, Art Owen, and Cun-Hui Zhang. 2012. One permutation hashing. *Advances in Neural Information Processing Systems (NeurIPS)* (2012), 3113–3121.
- [17] Wenqing Lin, Hemeng Wang, Haodong Deng, and Qingxiao Sun. 2024. ScalFrag: Efficient Tiled-MTTKRP with Adaptive Launching on GPUs. In *2024 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 335–345.
- [18] Guanxiang Liu and Hao Wu. 2024. Hp-csf: An gpu optimization method for cp decomposition of incomplete tensors. In *IFIP International Conference on Network and Parallel Computing (NPC)*. Springer, 77–90.
- [19] Morten Mørup, Lars Kai Hansen, and Sidse M Arnfred. 2008. Algorithms for sparse nonnegative Tucker decompositions. *Neural computation (Neural Comput.)* (2008), 2112–2131.
- [20] Israt Nisa, Jiajia Li, Aravind Sukumaran-Rajam, Prasant Singh Rawat, Sriram Krishnamoorthy, and Ponnuswamy Sadayappan. 2019. An efficient mixed-mode

- representation of sparse tensors. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, 1–25.
- [21] Sophocles J Orfanidis. 1995. *Introduction to signal processing*. Prentice-Hall, Inc.
 - [22] Eric T Phipps, Nicholas T Johnson, and Tamara G Kolda. 2023. Streaming generalized canonical polyadic tensor decompositions. In *Proceedings of the Platform for Advanced Scientific Computing Conference (PASC)*. ACM, 1–10.
 - [23] Xiangjun Qu, Lei Gong, Wenqi Lou, Qianyu Cheng, Xianglan Chen, Chao Wang, and Xuehai Zhou. 2024. AutoSparse: A Source-to-Source Format and Schedule Auto-Tuning Framework for Sparse Tensor Program. In *2024 IEEE 42nd International Conference on Computer Design (ICCD)*. IEEE, 504–512.
 - [24] Paul Resnick and Hal R Varian. 1997. Recommender systems. *Communications of the ACM (CACM)* (1997), 1–49.
 - [25] Muhammad A Sharaf, Deborah L Illman, and Bruce R Kowalski. 1986. *Chemometrics*. John Wiley & Sons.
 - [26] Nicholas D Sidiropoulos, Lieven De Lathauwer, Xiao Fu, Kejun Huang, Evangelos E Papalexakis, and Christos Faloutsos. 2017. Tensor decomposition for signal processing and machine learning. *IEEE Transactions on signal processing (TSP)* (2017), 3551–3582.
 - [27] Shaden Smith, Jee W. Choi, Jiajia Li, Richard Vuduc, Jongsoo Park, Xing Liu, and George Karypis. 2017. *FROSTT: The Formidable Repository of Open Sparse Tensors and Tools*. <http://frosts.io/>
 - [28] Shaden Smith, Kejun Huang, Nicholas D Sidiropoulos, and George Karypis. 2018. Streaming tensor factorization for infinite data sources. In *Proceedings of the 2018 SIAM international conference on data mining (SDM)*. SIAM, 81–89.
 - [29] Shaden Smith, Niranjan Ravindran, Nicholas D Sidiropoulos, and George Karypis. 2015. SPLATT: Efficient and parallel sparse tensor-matrix multiplication. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 61–70.
 - [30] Yongseok Soh, Patrick Flick, Xing Liu, Shaden Smith, Fabio Checconi, Fabrizio Petrini, and Jee Choi. 2021. High performance streaming tensor decomposition. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 683–692.
 - [31] Yongseok Soh, Ahmed E Helal, Fabio Checconi, Jan Laukemann, Jesmin Jahan Tithi, Teresa Ranadive, Fabrizio Petrini, and Jee W Choi. 2023. Dynamic Tensor Linearization and Time Slicing for Efficient Factorization of Infinite Data Streams. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 402–412.
 - [32] Yongseok Soh, Ramakrishnan Kannan, Piyush Sao, and Jee Choi. 2024. Accelerated Constrained Sparse Tensor Factorization on Massively Parallel Architectures. In *Proceedings of the 53rd International Conference on Parallel Processing (ICPP)*. ACM, 107–116.
 - [33] Sangjun Son, Yong-chan Park, Minyong Cho, and U Kang. 2022. DAO-CP: Data-Adaptive Online CP decomposition for tensor stream. *Plos one* (2022), e0267091.
 - [34] Qingxiao Sun, Yi Liu, Ming Dun, Hailong Yang, Zhongzhi Luan, Lin Gan, Guangwen Yang, and Depei Qian. 2020. Sptfs: Sparse tensor format selection for mttkrp via deep learning. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE, 1–14.
 - [35] Ryan Swann, Muhammad Osama, Karthik Sangaiah, and Jalal Mahmud. 2024. Seer: Predictive Runtime Kernel Selection for Irregular Problems. In *2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 133–142.
 - [36] Michiel Vandecappelle, Nico Vervliet, and Lieven De Lathauwer. 2017. Nonlinear least squares updating of the canonical polyadic decomposition. In *2017 25th European Signal Processing Conference (EUSIPCO)*. IEEE, 663–667.
 - [37] Sasindu Wijeratne, Rajgopal Kannan, and Viktor Prasanna. 2025. Accelerating Sparse MTTKRP for Small Tensor Decomposition on GPU. *arXiv preprint arXiv:2503.18198* (2025), 1–6.
 - [38] Jiaming Xu, Shan Huang, Jinhao Li, Guyue Huang, Yuan Xie, Yu Wang, and Guohao Dai. 2024. Enabling Efficient Sparse Multiplications on GPUs with Heuristic Adaptability. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* (2024), 2226 – 2239.
 - [39] Chaoqi Yang, Cheng Qian, and Jimeng Sun. 2022. GOCPT: Generalized Online Canonical Polyadic Tensor Factorization and Completion. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI)*. IJCAI Organization, 2348–2354.
 - [40] Keyu Yang, Yunjun Gao, Yifeng Shen, Baihua Zheng, and Lu Chen. 2023. DISTRASD: An efficient distributed multi-aspect streaming tensor decomposition. In *Proceedings of the ACM Turing Award Celebration Conference-China 2023 (Turing)*. ACM, 127–128.
 - [41] Serif Yesil, Azin Heidarshenas, Adam Morrison, and Josep Torrellas. 2023. Wise: Predicting the performance of sparse matrix vector multiplication with machine learning. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP)*. ACM, 329–341.
 - [42] Xi Zhang, Yanyi Li, Yisi Luo, Qi Xie, and Deyu Meng. 2025. Online Functional Tensor Decomposition via Continual Learning for Streaming Data Completion. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems (NeurIPS)*. NeurIPS Foundation, 10565–10593.
 - [43] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, et al. 2020. An-sor: Generating {High-Performance} tensor programs for deep learning. In *14th USENIX symposium on operating systems design and implementation (OSDI)*. USENIX Association, 863–879.
 - [44] Shuo Zhou, Nguyen Xuan Vinh, James Bailey, Yunzhe Jia, and Ian Davidson. 2016. Accelerating online cp decompositions for higher order tensors. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. ACM, 1375–1384.